

Django Caching, Cookies & Sessions

Complete Beginner to Advanced Notes

Part 1 -- Django Caching

1. What is Caching?

Caching means temporarily storing frequently used data so that it can be retrieved quickly instead of performing the same expensive operation repeatedly.

For example:

javascript.snippet

JAVASCRIPT

Database Query SlowAPI Request SlowComplex Calculation ExpensiveTemplate Rendering
Expensive

Without caching:

javascript.snippet

JAVASCRIPT

Request Database / API / Calculation Generate Result Response

With caching:

javascript.snippet

JAVASCRIPT

Request Cache Found Return Cached Data Not Found Database
/ API Store in Cache Response

Main benefits

javascript.snippet

JAVASCRIPT

Faster responseReduced database loadReduced repeated computationBetter scalabilityBetter user experience

2. Why Use Caching in Django?

Caching is useful when the same data or result is requested frequently.

Common examples:

```
javascript.snippet JAVASCRIPT

Product listProduct detailsHomepagePopular postsAPI responsesDashboard
statisticsExpensive
database queriesExpensive template fragments
```

Benefits

```
javascript.snippet JAVASCRIPT

1. Faster website2. Lower database load3. Lower CPU usage4. Better scalability5. Better
user experience
```

3. Types of Django Caching

The main caching approaches are:

```
javascript.snippet JAVASCRIPT

1. Per-site cache2. Per-view cache3. Template fragment cache4. Low-level cache API
```

4. Django Cache Backends

Django supports several cache backends.

4.1 LocMemCache

```
javascript.snippet JAVASCRIPT

Local Memory Cache
```

Data is stored in the memory of the Django process.

Example:

```
javascript.snippet JAVASCRIPT

CACHES = {    "default": {        "BACKEND": (
"django.core.cache.backends.locmem.LocMemCache"
    ),        "LOCATION": "unique-snowflake",    }}
```

Good for

```
javascript.snippet JAVASCRIPT

DevelopmentTestingSmall applicationsSimple local caching
```

Limitation

If multiple Django processes or servers are running, each process may have its own local cache.

Therefore, it is generally not suitable as the shared cache for a distributed production system.

5. File-Based Cache

File-based caching stores cached data on the filesystem.

Example:

```
javascript.snippet JAVASCRIPT

CACHES = {    "default": {        "BACKEND": (
"django.core.cache.backends.filebased.FileBasedCache"
    ),        "LOCATION": "/var/tmp/django_cache",    }}
```

Advantages

```
javascript.snippet JAVASCRIPT

Easy to configureDoes not require a separate cache server
```

Disadvantages

```
javascript.snippet JAVASCRIPT

Slower than memory-based cacheFilesystem overheadNot ideal for distributed systems
```

6. Database Cache

Django can also store cache data in a database.

Backend:

```
javascript.snippet
```

JAVASCRIPT

```
"django.core.cache.backends.db.DatabaseCache"
```

Example:

```
javascript.snippet
```

JAVASCRIPT

```
CACHES = {    "default": {        "BACKEND": (  
    "django.core.cache.backends.db.DatabaseCache"  
        ),        "LOCATION": "my_cache_table",    }}
```

You also need to create the cache table.

Example:

```
javascript.snippet
```

JAVASCRIPT

```
python manage.py createcachetable
```

Important

Database caching can be useful in some situations, but using your main database as a high-volume cache can increase database load.

7. Memcached

Memcached is a distributed, memory-based caching system.

Characteristics:

```
javascript.snippet
```

JAVASCRIPT

```
RAM-basedVery fastDistributedSimpleVolatile
```

It is commonly used when you need a simple distributed cache.

8. Redis

Redis is an in-memory data store that is commonly used for caching.

It can also be used for other purposes such as:

```
javascript.snippet
```

JAVASCRIPT

```
CachingSessionsQueuesPub/SubCountersDistributed locksRate limiting
```

Redis is commonly used as a production cache for Django applications.

Important

Redis is **not one of Django's built-in cache backends in the same sense as LocMemCache or DatabaseCache**.

A Redis backend/package is typically used to connect Django's cache framework to Redis.

9. Redis Setup

A commonly used package is:

```
javascript.snippet
```

JAVASCRIPT

```
pip install django-redis
```

Example:

```
javascript.snippet
```

JAVASCRIPT

```
CACHES = {      "default": {      "BACKEND": "django_redis.cache.RedisCache",  
"LOCATION"  
: "redis://127.0.0.1:6379/1",      "OPTIONS": {      "CLIENT_CLASS": (  
"django_redis.client.DefaultClient"      ),      },      },}
```

The exact Redis configuration depends on your deployment environment.

10. Per-View Cache

Per-view caching stores the response of a particular view.

Example:

```
javascript.snippet JAVASCRIPT

from django.views.decorators.cache import cache_page@cache_page(60 * 15)def my_view(
request
): return render( request, "my_template.html", {}, )
```

Here:

```
javascript.snippet JAVASCRIPT

60 × 15 = 900 seconds
```

So the response can be cached for:

```
javascript.snippet JAVASCRIPT

15 minutes
```

Best use cases

```
javascript.snippet JAVASCRIPT

Public pagesProduct listing pagesFrequently visited pagesPages whose data does not
change
frequently
```

11. Template Fragment Cache

Sometimes we do not want to cache the entire page.

Instead, we may cache only an expensive part of the template.

Example:

```
javascript.snippet JAVASCRIPT

{% load cache %}{% cache 600 sidebar %}           Expensive sidebar content    {%
endcache
%}
```

Here:

```
javascript.snippet JAVASCRIPT
600 seconds = 10 minutes
```

Only the specified template fragment is cached.

Example

Suppose a page contains:

```
javascript.snippet JAVASCRIPT
HeaderProduct ListPopular ProductsFooter
```

Maybe only:

```
javascript.snippet JAVASCRIPT
Popular Products
```

is expensive to generate.

We can cache that fragment instead of caching the entire page.

12. Low-Level Cache API

Django provides a low-level cache API.

Import:

```
javascript.snippet JAVASCRIPT
from django.core.cache import cache
```

12.1 Set Cache

```
javascript.snippet JAVASCRIPT
cache.set("my_key", "hello world", timeout=300,)
```

The value is cached for:

javascript.snippet

JAVASCRIPT

```
300 seconds = 5 minutes
```

12.2 Get Cache

javascript.snippet

JAVASCRIPT

```
value = cache.get("my_key")
```

If the key exists:

javascript.snippet

JAVASCRIPT

```
value "hello world"
```

If it does not exist:

javascript.snippet

JAVASCRIPT

```
None
```

is normally returned unless a default value is provided.

Example:

javascript.snippet

JAVASCRIPT

```
value = cache.get("my_key", "default value",)
```

12.3 Delete Cache

javascript.snippet

JAVASCRIPT

```
cache.delete("my_key")
```

12.4 Clear Cache

```
javascript.snippet JAVASCRIPT
cache.clear()
```

Warning

`cache.clear()` can remove all entries from the configured cache backend.

Use it carefully in production.

13. Cache Key

A **cache key** is a unique identifier used to store and retrieve cached data.

Example:

```
javascript.snippet JAVASCRIPT
cache.set(    f"product_{product_id}",    product_data,    300, )
```

For:

```
javascript.snippet JAVASCRIPT
product_id = 10
```

the key becomes:

```
javascript.snippet JAVASCRIPT
product_10
```

For:

```
javascript.snippet JAVASCRIPT
product_id = 20
```

the key becomes:

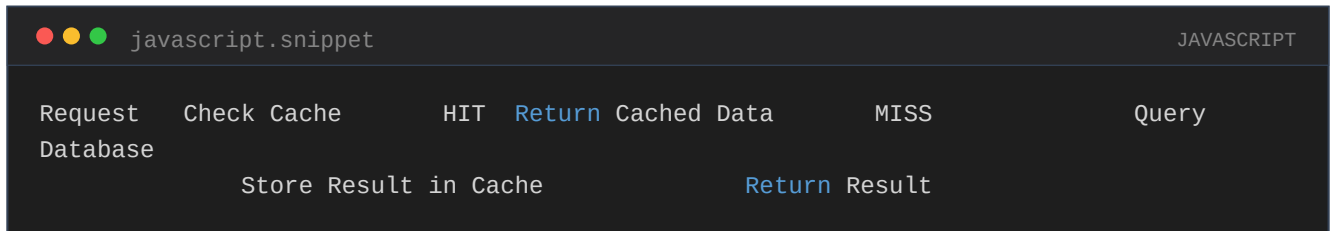
```
javascript.snippet JAVASCRIPT
product_20
```

Therefore, different products can have different cache entries.

14. Cache-Aside Pattern

Cache-Aside is one of the most commonly used caching strategies.

Flow:



Example:

```
product = cache.get(cache_key)if product is None:    product = Product.objects.get(        id=product_id    )    cache.set(        cache_key,        product,        300,    )
```

15. Cache Hit and Cache Miss

Cache Hit

When the requested data exists in cache:

```
Request    Cache    Data Found    Return Data
```

This is called a:

```
Cache Hit
```

Cache Miss

When the requested data does not exist:

javascript.snippetJAVASCRIPT

RequestCacheNot FoundDatabaseStore in Cache

This is called a:

javascript.snippetJAVASCRIPT

Cache Miss

16. Real-World Product Example

javascript.snippetJAVASCRIPT

```
from django.core.cache import cachedef product_detail(request, product_id):
    cache_key
    = f"product_{product_id}"    product = cache.get(cache_key)    if product is None:

    product = Product.objects.get(            id=product_id            )            cache.set(

    cache_key,            product,            300,            )    return render(
    request
    ,            "product.html",            {            "product": product,            },            )
```

Flow:

javascript.snippetJAVASCRIPT

Request Product 10Check product_10Cache? / \ Yes

NoReturnDatabaseCache Set

Return

17. Cache Invalidation

One of the most important caching concepts is:

javascript.snippetJAVASCRIPT

Cache Invalidation

When the original data changes, the cached version may become outdated.

Example:

```
javascript.snippet JAVASCRIPT
Database:Product price = $:Product price = $100
```

Suppose the database is updated:

```
javascript.snippet JAVASCRIPT
Database:Product price = $120
```

But cache still contains:

```
javascript.snippet JAVASCRIPT
$100
```

The user may receive stale data.

Therefore, after updating the data, the cache may need to be invalidated.

Example:

```
javascript.snippet JAVASCRIPT
cache.delete(    f"product_{product_id}")
```

18. Cache Timeout

A timeout defines how long a cache entry should remain valid.

Example:

```
javascript.snippet JAVASCRIPT
cache.set(    "product_10",    product,    timeout=300,)
```

Here:

```
javascript.snippet JAVASCRIPT
300 seconds = 5 minutes
```

After the timeout expires, the cache entry can no longer be relied upon.

Choosing timeout

Frequently changing data:

```
javascript.snippet JAVASCRIPT
Short timeout
```

Rarely changing data:

```
javascript.snippet JAVASCRIPT
Longer timeout
```

19. Cache Strategies

Cache-Aside

```
javascript.snippet JAVASCRIPT
Read Cache  Miss?  Read Database  Set Cache
```

Very common in web applications.

Write-Through

When data is written, the database and cache are updated together.

Concept:

```
javascript.snippet JAVASCRIPT
Write Request  Database  +Cache
```

This can keep cached data fresher but introduces additional write complexity.

20. Cache Versioning

Django's cache API supports cache versions.

Example:

```
javascript.snippet JAVASCRIPT
cache.set(    "key",    "value",    timeout=300,    version=2,)
```

Then:

```
javascript.snippet JAVASCRIPT
cache.get(    "key",    version=2,)
```

Cache versioning can help separate different versions of cached data.

21. get_or_set()

Django provides:

```
javascript.snippet JAVASCRIPT
cache.get_or_set()
```

Example:

```
javascript.snippet JAVASCRIPT
value = cache.get_or_set(    "key",    "value",    60,)
```

If the key exists:

```
javascript.snippet JAVASCRIPT
Return existing value
```

If it does not exist:

```
javascript.snippet JAVASCRIPT

Store the default valueReturn it
```

For expensive database queries, the default value can also be generated dynamically.

22. Common Caching Mistakes

1. Forgetting Cache Invalidation

```
javascript.snippet JAVASCRIPT

Database updated      Cache not updated/deleted      Stale data
```

2. Caching Everything

Not every piece of data should be cached.

Over-caching can cause:

```
javascript.snippet JAVASCRIPT

Memory usageStale dataComplex invalidationUnnecessary complexity
```

3. Wrong Timeout

Too short:

```
javascript.snippet JAVASCRIPT

Low cache hit rate
```

Too long:

```
javascript.snippet JAVASCRIPT

Stale data
```

4. Caching Sensitive Data

Avoid blindly caching sensitive information such as:

 javascript.snippet

JAVASCRIPT

PasswordsAuthentication secretsAccess tokensPrivate user informationPayment information

23. When Should You Use Caching?

Good candidates include:

 javascript.snippet

JAVASCRIPT

Frequently requested dataExpensive database queriesExpensive calculationsPublic API responsesProduct listsProduct detailsHomepage dataDashboard statisticsTemplate fragments

24. When Should You Avoid Caching?

Caching may be inappropriate when:

 javascript.snippet

JAVASCRIPT

Data changes extremely frequentlyData must always be real-timeThe data is highly sensitiveThe computation is already very cheapCache invalidation is more expensive than the operation itself

25. Production Caching Recommendation

For a typical Django e-commerce application:

```
javascript.snippet JAVASCRIPT

Redis Shared production cachePer-view cache Public/list pagesLow-level cache
Specific
expensive dataTemplate fragment cache Expensive template sections
```

26. Caching Best Practices

```
javascript.snippet JAVASCRIPT

1. Use meaningful cache keys.2. Choose appropriate timeouts.3. Invalidate stale data.4.
Do not cache sensitive data blindly.5. Monitor cache hit/miss behavior.6. Avoid caching
everything.7. Use a shared cache for multi-server deployments.8. Keep cache-related
logic
simple.9. Consider cache stampede protection for expensive data.10. Test behavior when
the cache is unavailable.
```

Part 2 -- Django Cookies

27. What is a Cookie?

A **cookie** is a small piece of data stored by the browser and sent with requests to the appropriate website.

Cookies are primarily **client-side storage**.

Example:

```
javascript.snippet JAVASCRIPT

Browser Cookie Website
```

Cookies can be used for:

```
javascript.snippet JAVASCRIPT

PreferencesSession identifiersLanguage selectionSmall client-side stateTracking-related
functionality
```

Important

Cookies have limited size, commonly around **4 KB per cookie**, depending on browser and specification limits.

Sensitive information should not be stored directly in cookies unless the security design explicitly requires it and appropriate protections are used.

28. Setting a Cookie in Django

```
javascript.snippet JAVASCRIPT

from django.http import HttpResponseRedirect
def set_cookie(request):
    response = HttpResponseRedirect(
        "Cookie set"
    )
    response.set_cookie("username", "Faruk",
    )
    return response
```

The browser receives a Set-Cookie response header.

29. Reading a Cookie

```
javascript.snippet JAVASCRIPT

from django.http import HttpResponseRedirect
def get_cookie(request):
    username = request.COOKIES
    .get("username")
    return HttpResponseRedirect(
        username or "No cookie found"
    )
```

Django provides incoming cookies through:

```
javascript.snippet JAVASCRIPT

request.COOKIES
```

30. Deleting a Cookie

```
javascript.snippet JAVASCRIPT

from django.http import HttpResponseRedirect
def delete_cookie(request):
    response = HttpResponseRedirect(
        "Cookie deleted" )
    response.delete_cookie("username")
    return response
```

31. Important Cookie Security Attributes

Important cookie attributes include:

```
javascript.snippet JAVASCRIPT

HttpOnlySecureSameSiteMax-AgeExpiresPathDomain
```

HttpOnly

```
javascript.snippet JAVASCRIPT

HttpOnly
```

prevents normal JavaScript code from reading the cookie.

It helps reduce certain types of cookie theft through XSS, but it does **not** make the application completely secure.

Secure

```
javascript.snippet JAVASCRIPT

Secure
```

means the cookie should only be sent over HTTPS.

SameSite

SameSite controls when browsers send cookies in cross-site contexts.

Common values:

```
javascript.snippet JAVASCRIPT

StrictLaxNone
```

The correct choice depends on the application's requirements.

32. Example of a More Secure Cookie

```
javascript.snippet JAVASCRIPT

response.set_cookie("username", "Faruk", httponly=True, secure=True,
samesite
="Lax",)
```

For local development without HTTPS, `secure=True` may prevent the cookie from being sent.

Part 3 -- Django Sessions

33. What is a Session?

A **session** allows Django to store user-specific state across multiple requests.

A typical Django session uses:

```
javascript.snippet JAVASCRIPT

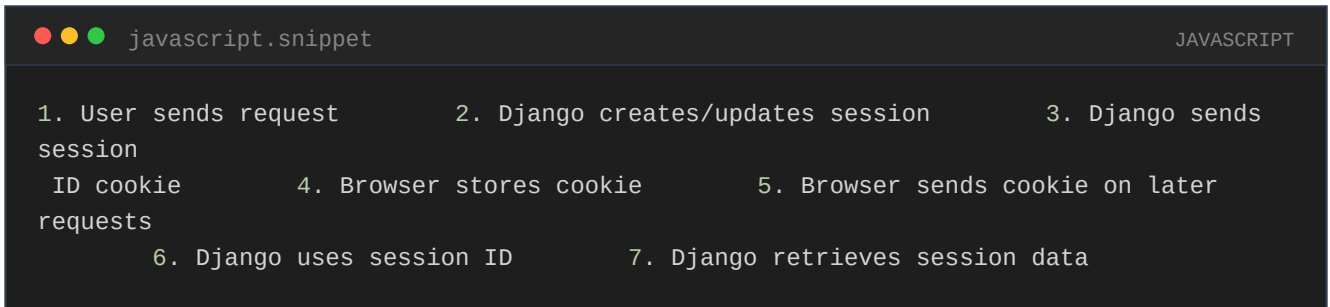
Browser    Session ID Cookie    Django    Session Data
```

The browser normally stores a session identifier, while the actual session data is stored by the configured Django session backend.

Therefore, a session is different from simply storing all data directly in a cookie.

34. How Django Sessions Work

Typical flow:



35. Where Does Django Store Sessions?

By default, Django uses the database-backed session engine when sessions are enabled with the standard configuration.

The session table is typically:

```
django_session
```

Other session backends can be configured, including:

```
DatabaseCacheCached databaseFileSigned cookies
```

The exact behavior depends on the configured session engine.

36. Setting a Django Session

```
from django.http import HttpResponseRedirectdef set_session(request):    request.session["username"] = "Faruk"    request.session["role"] = "admin"    return HttpResponseRedirect("/")
```

37. Getting Session Data

```
javascript.snippet JAVASCRIPT

from django.http import HttpResponseRedirect
def get_session(request):
    username = request.session.get("username")
    return HttpResponseRedirect(username or "No username")
)
```

38. Deleting Session Data

To remove one session key:

```
javascript.snippet JAVASCRIPT

del request.session["username"]
```

A safer alternative:

```
javascript.snippet JAVASCRIPT

request.session.pop("username", None,)
```

39. Clearing the Entire Session

To completely clear the current session:

```
javascript.snippet JAVASCRIPT

request.session.flush()
```

This removes the current session data and creates a new session key.

40. Session Expiry

Set session expiry in seconds:

```
javascript.snippet JAVASCRIPT

request.session.set_expiry(300)
```

This means:

```
javascript.snippet
300 seconds = 5 minutes
```

JAVASCRIPT

Browser-Close Session

To make the session expire when the browser is closed:

```
javascript.snippet
request.session.set_expiry(0)
```

JAVASCRIPT

This uses a browser-session cookie.

41. Django Authentication and Sessions

Django authentication commonly uses sessions to maintain login state.

Example:

```
javascript.snippet
from django.contrib.auth import authenticate, login
username = input('Username: ')
password = input('Password: ')
user = authenticate(username=username, password=password)
if user is not None:
    login(request, user)
```

JAVASCRIPT

The `login()` function establishes the authenticated session.

Later, the user can be accessed through:

```
javascript.snippet
request.user
```

JAVASCRIPT

Part 4 -- Cookies vs Sessions

42. Cookie vs Session

Feature	Cookie	Session
Main storage	Browser	Server-side/session backend
Typical client data	Cookie value	Session ID
Size	Limited	Generally much larger
Sensitive data	Avoid storing directly	Better suited, but still requires security
Server-side state	No	Yes
Common use	Preferences, identifiers	Login state, cart, user state
Django API	request.COOKIES	request.session

Important

Saying:

```

javascript.snippet
JAVASCRIPT

Cookie = insecureSession = completely secure

```

is incorrect.

Both require proper security configuration.

43. Session Cookie in Django

A Django session normally uses a cookie such as:

```

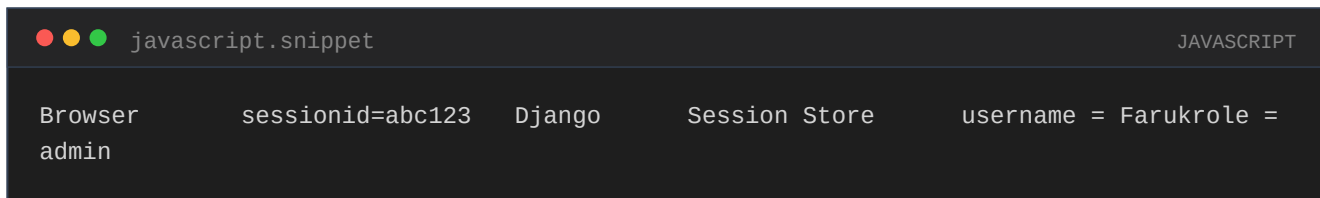
javascript.snippet
JAVASCRIPT

sessionid

```

The cookie typically contains the session identifier rather than the complete session data when using Django's normal server-side session backends.

Conceptually:

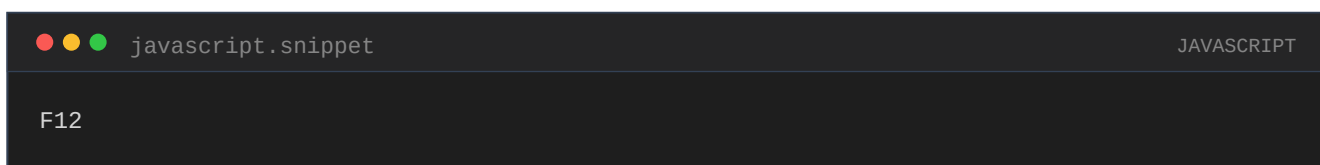


44. How to Inspect Django Session Cookies in Chrome

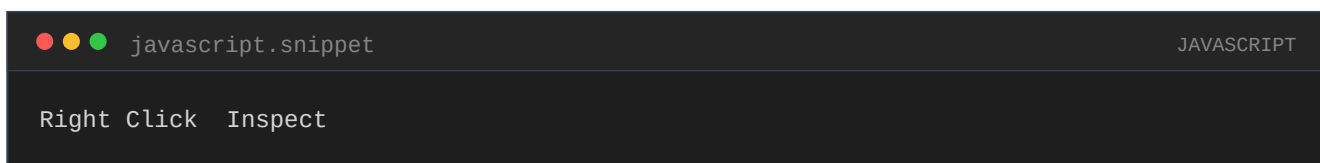
You can inspect cookies using Chrome DevTools.

Step 1 -- Open DevTools

Press:

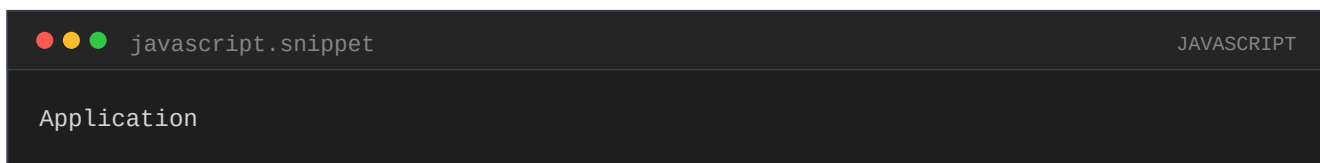


or:

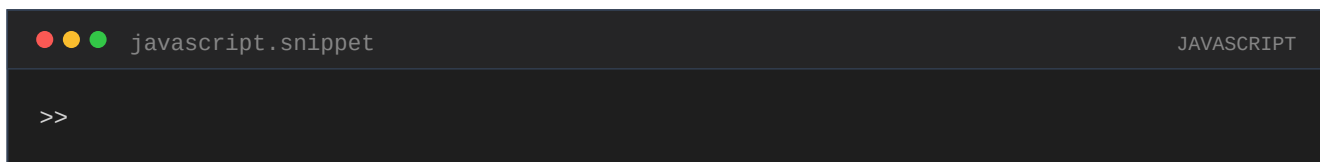


Step 2 -- Open Application

Select:



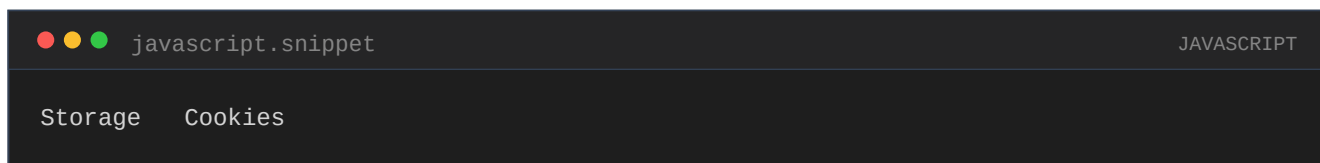
If you cannot see it, click the:



menu.

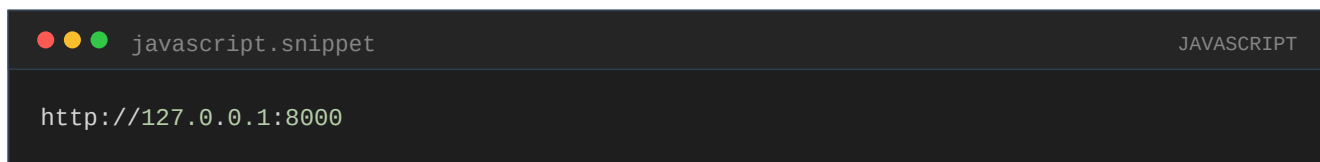
Step 3 -- Find Cookies

In the left sidebar:



Step 4 -- Select Your Website

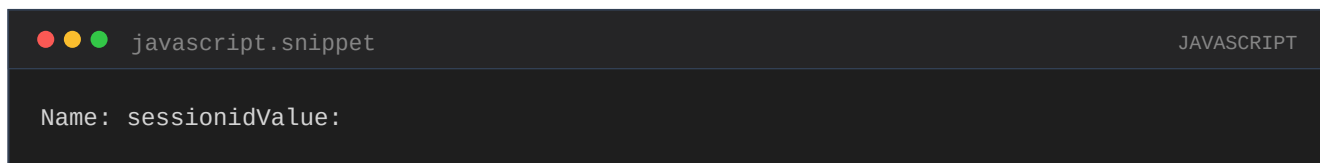
For local development, you may see something like:



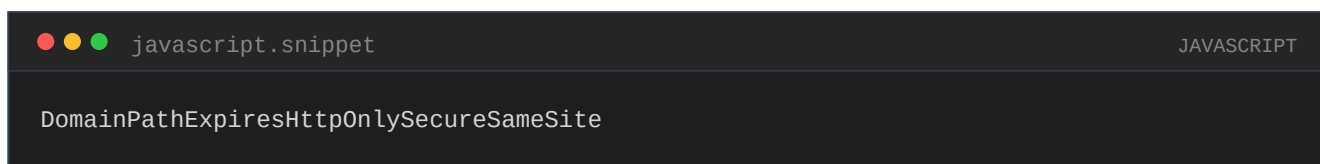
Click the domain.

Step 5 -- Inspect sessionid

You may see:



Other columns may include:



Important

The `sessionid` value is a credential-like identifier. Do not share it publicly.

Also, the value does **not** normally contain the actual session data when using Django's standard server-side session backends.

45. Complete Session Flow



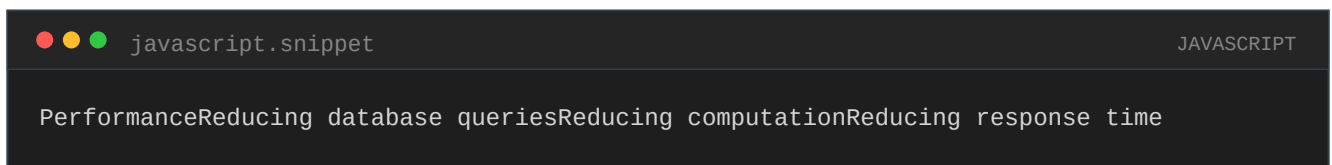
Part 5 -- Caching vs Sessions

46. Cache vs Session

These two concepts are different.

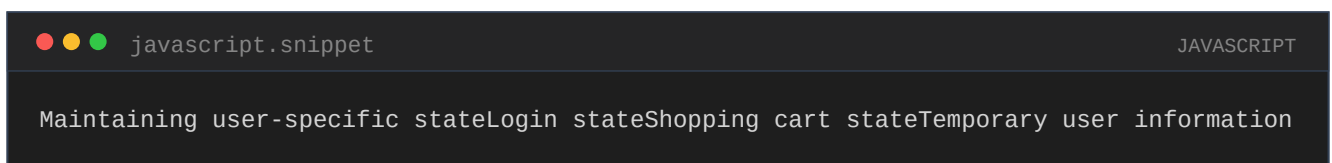
Cache

Used mainly for:

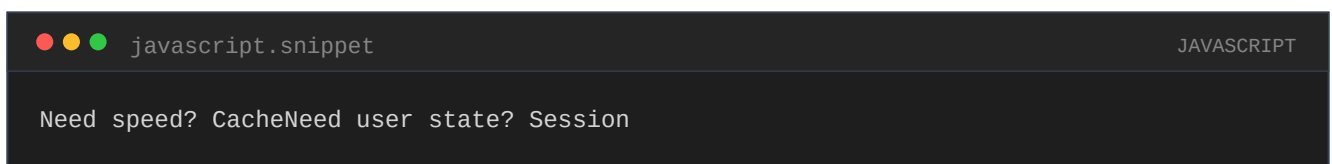


Session

Used mainly for:



Simple Rule



47. Cache vs Session Example

Suppose we have an e-commerce website.

Product List

```
javascript.snippet JAVASCRIPT

Frequently requestedNot user-specificExpensive database query
```

Use:

```
javascript.snippet JAVASCRIPT

Cache
```

Shopping Cart

```
javascript.snippet JAVASCRIPT

User-specificChanges frequentlyAssociated with a particular user
```

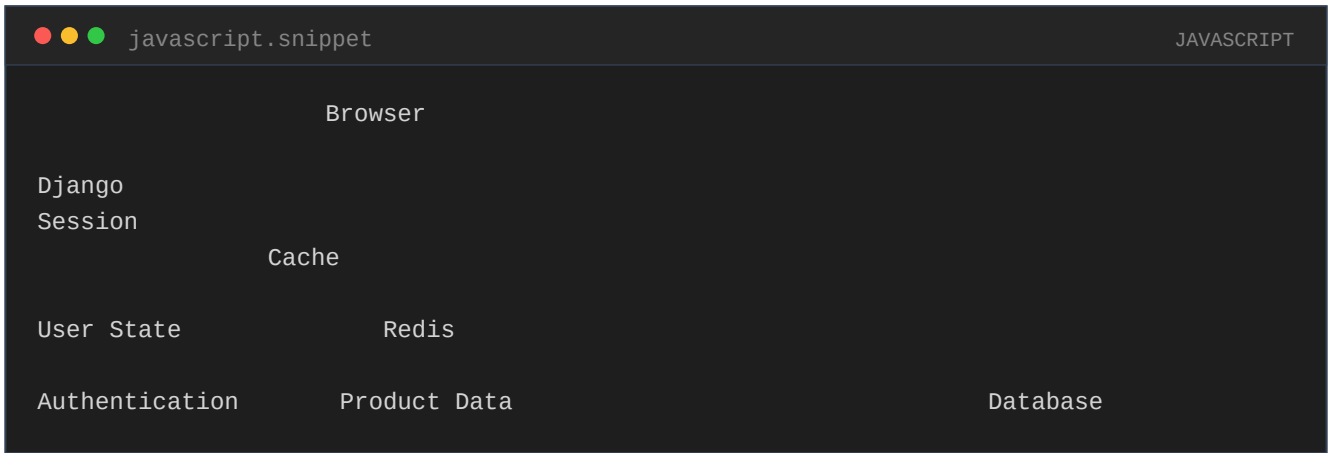
Use:

```
javascript.snippet JAVASCRIPT

Session or database/cart system
```

Part 6 -- Real-World E-Commerce Architecture

48. Example Architecture



Example:

```
javascript.snippet JAVASCRIPT

Login SessionShopping Cart Session / DatabaseProduct List CacheProduct Details
CacheExpensive
Query Low-Level CacheExpensive Template Section Template Fragment Cache
```

49. Example: Product Detail with Cache

```
javascript.snippet JAVASCRIPT

from django.core.cache import cachefrom django.shortcuts import get_object_or_404from
django
.shortcuts import renderdef product_detail( request, product_id,): cache_key =
( f"product_{product_id}" ) product = cache.get( cache_key )
if
product is None: product = get_object_or_404( Product, id
=
product_id, ) cache.set( cache_key, product,
timeout=300, ) return render( request, "product.html", {
"product": product, }, )
```

50. Invalidate Product Cache

Suppose the product is updated:

```
javascript.snippet JAVASCRIPT

product.price = .save()
```

Then invalidate the cache:

```
javascript.snippet JAVASCRIPT

cache.delete( f"product_{product.id}")
```

Next request:

```
javascript.snippet JAVASCRIPT
Cache MISS Database New Product Data Cache SET
```

51. Important Production Considerations

Cache Failure

Your application should ideally be designed so that an unavailable cache does not unnecessarily make the entire application unusable.

For example:

```
javascript.snippet JAVASCRIPT
Cache unavailable Fallback to database
```

The exact strategy depends on the application.

Cache Stampede

Suppose a very popular cache entry expires.

Hundreds of requests may simultaneously discover:

```
javascript.snippet JAVASCRIPT
Cache MISS
```

and all query the database.

This can create a sudden load spike.

This problem is commonly called:

```
javascript.snippet JAVASCRIPT
Cache Stampede
```

Possible solutions include:

```
LockingRequest coalescingStaggered expirationBackground refreshAppropriate cache  
strategy
```

52. Important Terms

Term	Meaning
Cache	Temporary stored data for faster access
Cache Hit	Data found in cache
Cache Miss	Data not found in cache
Cache Key	Unique identifier for cached data
Timeout	Cache lifetime
Invalidation	Removing/updating stale cache
Cache Backend	System used to store cache
Redis	In-memory data store commonly used for caching
Session	User-specific state across requests
Cookie	Small browser-stored piece of data
Session ID	Identifier used to locate a user's session
Cache Stampede	Many requests rebuilding an expired cache simultaneously

53. Interview Questions

Q1. What is caching?

Caching is the process of temporarily storing frequently accessed data so that future requests can retrieve it faster without repeating expensive operations.

Q2. Why do we use caching?

Faster responseReduced database loadReduced computationBetter scalability

Q3. What are the main Django caching approaches?

Per-site cachePer-view cacheTemplate fragment cacheLow-level cache API

Q4. What is a cache backend?

A cache backend is the storage system used to store cached data.

Examples:

LocMemCacheFileBasedCacheDatabaseCacheMemcachedRedis

Q5. What is a cache hit?

When requested data is found in the cache.

Q6. What is a cache miss?

When requested data is not found in the cache.

Q7. What is cache invalidation?

Removing or updating cached data when the underlying source data changes.

Q8. What is Cache-Aside?

A caching strategy where the application:

```
javascript.snippet JAVASCRIPT
Checks cache If miss Reads database Stores result in cache
```

Q9. What is a cookie?

A small piece of data stored by the browser and sent with requests to the relevant website.

Q10. What is a Django session?

A mechanism for maintaining user-specific state across multiple requests.

Q11. Where is Django session data stored?

By default, Django commonly uses the database-backed session engine, but other session backends can be configured.

Q12. What is sessionId?

It is the cookie commonly used by Django to identify the current session when using the standard session middleware and cookie-based session identifier.

Q13. Cookie vs Session?

```
javascript.snippet JAVASCRIPT
Cookie Browser-side dataSession Server-side user state identified by a session cookie
```

Q14. What does HttpOnly do?

It prevents normal client-side JavaScript from accessing the cookie.

It does not make the cookie or application completely secure.

Q15. What does Secure do?

It instructs the browser to send the cookie only over HTTPS.

54. Quick Revision

Caching

```
javascript.snippet JAVASCRIPT
Cache Improve Performance Reduce Database Load
```

Main approaches:

```
javascript.snippet JAVASCRIPT
Per-sitePer-viewTemplate FragmentLow-level API
```

Common production backend:

```
javascript.snippet JAVASCRIPT
RedisMemcached
```

Cookie

```
javascript.snippet JAVASCRIPT
Browser Cookie Website
```

Common uses:

```
javascript.snippet JAVASCRIPT
PreferencesIdentifiersSession ID
```

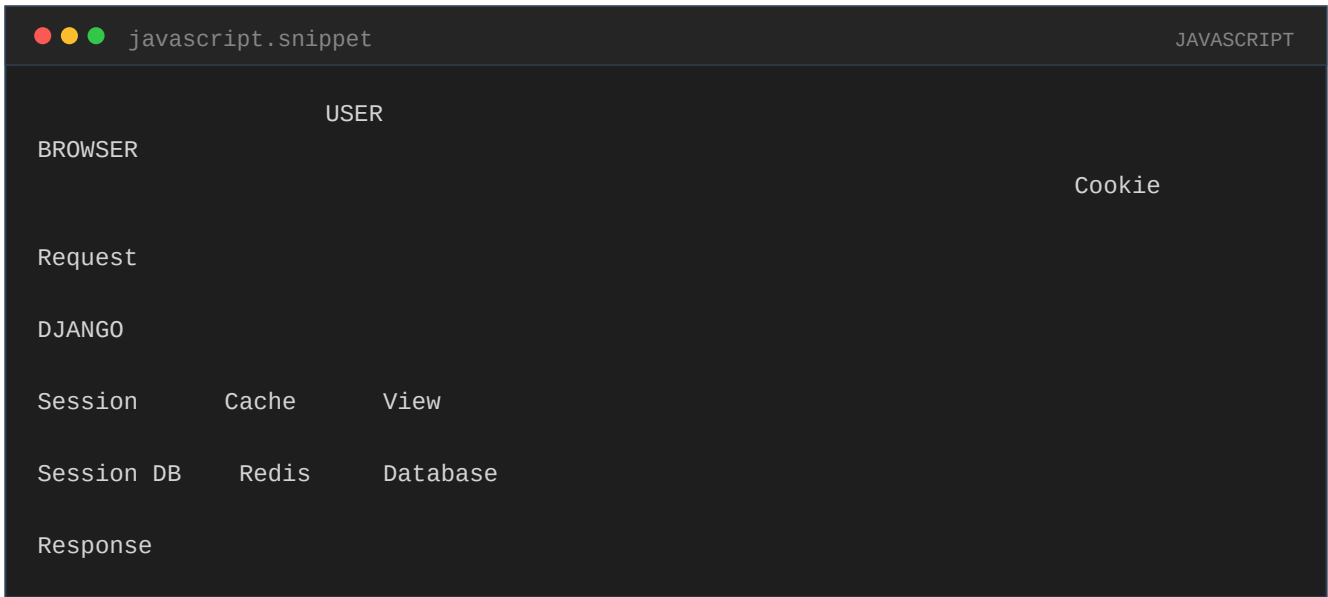
Session

```
javascript.snippet JAVASCRIPT
Browser Session ID Cookie Django Session Store
```

Common uses:

```
javascript.snippet JAVASCRIPT
LoginShopping CartUser StateTemporary Data
```

55. Final Mental Model



Final Rules to Remember

javascript.snippet

JAVASCRIPT

Caching Used mainly for performance.Cookie Small data stored in the browser.Session User
-specific state maintained across requests.Redis Common production cache backend.Cache Hit
Data found in cache.Cache Miss Data not found in cache.Cache Invalidation Remove/
update
stale cached data.Cache-Aside Check cache DB on miss Set cache.Per-view cache Cache
an
entire view response.Template fragment cache Cache only part of a template.Low-level
cache
API Direct control over cache data.sessionid Usually identifies the user's Django
session
.HttpOnly Prevents normal JavaScript access to a cookie.Secure Cookie is sent only over
HTTPS.

Golden Rule

javascript.snippet

JAVASCRIPT

Need better performance?
SESSIONNeed
small browser-side data?

CACHENeed user-specific state?

COOKIE